Program, with the slice of `q[4]`

		hop				srcSlice
		0	1	2	3	qc
3	`hops = 4`					
4	`q = QuantumRegister(1 + 2 * hops, "q")`					
5	`c = ClassicalRegister(2 * hops, "c")`					
6	`out = ClassicalRegister(1, "out")`					
7	`qc = QuantumCircuit(q, c, out)`					
8	`qc.u(0.4, 0.2, 0.1, q[0])`					■
9	`src = 0`					
10	`for h in range(hops):`					
11	`    a, b = 1 + 2 * h, 2 + 2 * h`					
12	`    qc.h(q[a])`	●	●	●	○	■
13	`    qc.cx(q[a], q[b])`	●	●	●	○	■
14	`    qc.cx(q[src], q[a])`	●	●	●	○	■
15	`    qc.h(q[src])`	●	●	●	○	■
16	`    qc.measure(q[src], c[2 * h])`	●	●	●	○	■
17	`    qc.measure(q[a], c[2 * h + 1])`	●	●	○	○	■
18	`    with qc.if_test((c[2 * h + 1], 1)):`					
19	`        qc.x(q[b])`	●	●	○	○	■
20	`    with qc.if_test((c[2 * h], 1)):`					
21	`        qc.z(q[b])`	●	●	○	○	■
22	`    src = b`					
23	`qc.measure(q[src], out[0])`					■

operations in the slice of `q[4]` — classical statement that computes them

● hop in the slice ○ hop left out ■ in srcSlice's slice of qc